

AWS DVA-C02: Bộ Đề Set 4 (Tổng Hợp Chi Tiết)

Phân tích chuyên sâu đáp án đúng & Giải thích chi tiết các phương án sai

Tổng cộng: 14 Câu hỏi Chuyên sâu

Câu 1: Tùy biến Lambda Runtime & Execution Environment

Một nhà phát triển đang xây dựng ứng dụng giám sát giao thông dựa trên AI sử dụng AWS Lambda. Do độ phức tạp, cần điều chỉnh cách Lambda chạy code khởi tạo (setup code) và cách đọc các sự kiện gọi hàm (invocation events) từ Lambda Runtime API. Cần sử dụng tính năng nào của Lambda?

✓ ĐÁP ÁN ĐÚNG

Custom Runtime

💡 GIẢI THÍCH CHI TIẾT ĐÁP ÁN ĐÚNG:

Khi bạn muốn sử dụng ngôn ngữ không hỗ trợ sẵn hoặc muốn can thiệp trực tiếp vào vòng lặp thực thi (Runtime Loop), quản lý file `bootstrap` để đọc event từ Lambda Runtime HTTP API (`/runtime/invocation/next`) và tự tùy biến phase khởi tạo (Initialization), bạn bắt buộc phải dùng **Custom Runtime**.

✗ Tại sao các phương án còn lại SAI?

- **Lambda Layers:** Chỉ dùng để chia sẻ file thư viện phụ thuộc (dependencies) hoặc data giữa các hàm Lambda để giảm dung lượng file deployment package, không thể can thiệp vào vòng lặp Runtime API.
- **Lambda@Edge:** Dùng để chạy code xử lý HTTP request/response tại các Edge Location của CloudFront, không liên quan đến Runtime API.
- **DLQ (Dead Letter Queue):** Dùng để hứng các event bị lỗi sau khi retry thất bại trong Async invocation, không quản lý môi trường thực thi.

Câu 2: Lập lịch cho ECS Task (Task Placement Strategy)

Cấu hình task definition trong ECS Cluster để đảm bảo các task được lập lịch trên các instance có đủ tài nguyên và tuân thủ các ràng buộc (implicit/explicit constraints). Phương án nào thỏa mãn với ÍT CẤU HÌNH NHẤT (LEAST amount of configuration)?

✓ ĐÁP ÁN ĐÚNG

Default Task Placement Strategy (Không chỉ định Placement Strategy thủ công)

💡 GIẢI THÍCH CHI TIẾT ĐÁP ÁN ĐÚNG:

Mặc định, ECS Task Scheduler sẽ tự động lọc các EC2 Instances thỏa mãn toàn bộ Implicit Constraints (đủ CPU/Memory) và Explicit Constraints. Sau đó, nó áp dụng chiến lược mặc định (như `binpack` hoặc `random` tùy loại service) mà **không yêu cầu bạn phải viết thêm bất kỳ đoạn cấu hình JSON/YAML nào**.

✗ Tại sao các phương án khác SAI / Không tối ưu?

- **Chiến lược `spread` thủ công:** Phải cấu hình rải đều task qua các AZs, tốn thêm công sức cấu hình.
- **Chiến lược `binpack` thủ công:** Phải định nghĩa quy tắc gom nhóm theo CPU/Memory, không phải là cách tốn ít cấu hình nhất khi chỉ cần đảm bảo đủ tài nguyên.

Câu 3: Đồng bộ trạng thái Game & Dữ liệu Cộng tác Real-Time

Phát triển game online cần đồng bộ app preferences và game state xuyên thiết bị, đồng thời cho phép nhiều người dùng đồng bộ và cộng tác dữ liệu dùng chung theo thời gian thực (real-time). Giải pháp nào phù hợp nhất?

✓ ĐÁP ÁN ĐÚNG

AWS AppSync (kết hợp GraphQL Subscriptions / Amplify DataStore)

💡 GIẢI THÍCH CHI TIẾT ĐÁP ÁN ĐÚNG:

AWS AppSync hỗ trợ **GraphQL Subscriptions (WebSockets)** giúp push dữ liệu real-time tới hàng triệu client, đồng thời hỗ trợ offline data sync và tự động giải quyết xung đột (conflict resolution) qua Amplify DataStore cho dữ liệu dùng chung giữa nhiều người chơi.

✗ Tại sao các dịch vụ khác SAI?

- **Amazon Cognito Sync (Legacy):** Chỉ dùng để sync các cặp Key-Value dung lượng nhỏ (< 1MB) cho cấu hình cá nhân của duy nhất 1 user, không hỗ trợ multi-user real-time collaboration.
- **Amazon S3:** Không có cơ chế real-time push subscription, độ trễ cao khi sync game state.
- **DynamoDB thuần:** Không tích hợp sẵn layer WebSocket real-time subscription cho client nếu không kết hợp AppSync/API Gateway.

Câu 4: Framework triển khai Serverless Stack (Lambda + API Gateway + DynamoDB)

Xây dựng kiến trúc Serverless mới gồm Lambda, API Gateway và DynamoDB trong 1 stack. Cần một framework đơn giản, tin cậy cho phép chia sẻ cấu hình chung (memory, timeout) và deploy toàn bộ tài nguyên thành 1 versioned entity duy nhất.

✓ ĐÁP ÁN ĐÚNG

AWS Serverless Application Model (AWS SAM)

💡 GIẢI THÍCH CHI TIẾT ĐÁP ÁN ĐÚNG:

AWS SAM mở rộng từ CloudFormation, cung cấp phần **Globals** giúp khai báo biến môi trường, memory, timeout chung cho toàn bộ Lambda functions. SAM đóng gói cả stack thành 1 CloudFormation template duy nhất với cú pháp ngắn gọn.

✗ Tại sao các dịch vụ khác SAI?

- **AWS CloudFormation thuần:** Cú pháp dài dòng hơn nhiều và **không có phần Globals** để chia sẻ cấu hình gọn nhẹ như SAM.
- **AWS CodeDeploy:** Dịch vụ quản lý quá trình triển khai (Canary/Linear deployment), không phải là framework định nghĩa hạ tầng (IaC).
- **AWS Amplify:** Hướng tới Full-stack mobile/web app frontend, không tối ưu bằng SAM cho cấu hình backend Serverless IaC thuần túy.

Câu 5: Tái sử dụng Kết nối Database trong AWS Lambda

Hàm Node.js trên Lambda mở kết nối tới RDS và đóng lại mỗi lần chạy. Làm cách nào để tái sử dụng kết nối CSDL đã tồn tại thay vì khởi tạo lại mỗi lần?

✓ ĐÁP ÁN ĐÚNG

Khai báo/Khởi tạo kết nối bên ngoài hàm Handler (Global Scope) / Amazon RDS Proxy

💡 GIẢI THÍCH CHI TIẾT ĐÁP ÁN ĐÚNG:

AWS Lambda tái sử dụng **Execution Context** giữa các lần gọi hàm (Warm Start). Việc đưa đoạn code khởi tạo DB connection ra ngoài `exports.handler` giúp nó chỉ chạy 1 lần trong phase Init, giúp các lượt invoke sau dùng lại connection có sẵn.

✗ Tại sao cách làm cũ SAI?

- **Khởi tạo bên trong Handler:** Bắt Lambda phải handshake lại với RDS ở mỗi request, gây tăng latency và làm sập connection pool của RDS khi scale-out.

Câu 6: Che thông tin PII trên S3 theo Role người dùng (Select THREE)

Lưu bản sao duy nhất file PII trên S3 trong 5 năm. Cần che dữ liệu (redact) tùy theo role người dùng khi họ đọc dữ liệu. Chọn 3 hành động đúng.

✓ 3 ĐÁP ÁN ĐÚNG

1. Create an S3 Access Point for each user role.
2. Configure an S3 Object Lambda Access Point for each S3 Access Point name. Associate the RedactPII-[role] Lambda function.
3. Use the GetObject API to retrieve the redacted data.

💡 GIẢI THÍCH CHI TIẾT ĐÁP ÁN ĐÚNG:

S3 Object Lambda chèn Lambda function vào giữa request `GetObject` để biến đổi/che dữ liệu on-the-fly trước khi trả về client. Giữ nguyên 1 file gốc duy nhất trên S3.

✗ Tại sao các phương án khác SAI?

- **S3 Event Notifications khi GET:** Event notification chạy bất đồng bộ (async), **không thể can thiệp luồng GET đồng bộ** để biến đổi data trả về.
- **S3 Replication:** Tạo ra nhiều bản sao bucket, vi phạm yêu cầu "Maintain a single copy".
- **GetObjectLegalHold API:** Dùng để check trạng thái khóa chống xóa file, không dùng để đọc nội dung file.

Câu 7: Mã hóa S3 At-Rest với Khóa do Công ty tự quản lý (Select TWO)

Lưu dữ liệu S3 từ EC2 qua HTTPS. Cần mã hóa at-rest bằng AES-256 với chìa khóa do chính công ty cung cấp và tự quản lý (provided and managed by the company).

✓ 2 ĐÁP ÁN ĐÚNG

1. Implement Amazon S3 server-side encryption with customer-provided keys (SSE-C).
2. Encrypt the data on the client-side before sending to Amazon S3 using their own master key.

💡 GIẢI THÍCH CHI TIẾT ĐÁP ÁN ĐÚNG:

Cả **SSE-C** và **Client-Side Encryption** đều cho phép công ty nắm giữ hoàn toàn Master Key trên hạ tầng riêng (AWS không lưu trữ hay quản lý khóa này).

✗ Tại sao các phương án còn lại SAI?

- **SSE-S3:** Khóa do AWS tự tạo và quản lý 100%.
- **SSE-KMS:** Khóa vẫn do dịch vụ AWS KMS lưu trữ và quản lý hạ tầng HSM.
- **Dùng SSL/TLS:** Chỉ mã hóa trên đường truyền (In-Transit), không phải At-Rest.

Câu 8: Quản lý & Tự động Xoay vòng Mật khẩu RDS

Bảo mật database credentials, API keys dùng trong RDS. Cần mã hóa, xoay vòng định kỳ (rotated regularly) và ứng dụng tự nhận bản mới nhất khi kết nối.

✓ ĐÁP ÁN ĐÚNG

Use AWS Secrets Manager to store and encrypt credentials and enable automatic rotation.

💡 GIẢI THÍCH CHI TIẾT ĐÁP ÁN ĐÚNG:

AWS Secrets Manager tích hợp sẵn tính năng **Automatic Rotation** cho RDS thông qua Lambda mẫu có sẵn. Khi app gọi `GetSecretValue`, nó sẽ luôn lấy phiên bản mật khẩu mới nhất (`AWSCURRENT`).

✗ Tại sao Systems Manager Parameter Store SAI?

- **SSM Parameter Store (SecureString)**: Mặc dù mã hóa được bằng KMS nhưng **không có tính năng tự động xoay vòng mật khẩu RDS sẵn (No native automatic rotation)**, phải tự viết code rất phức tạp.

Câu 9: Tích hợp LDAP không hỗ trợ SAML 2.0 với AWS IAM

Tích hợp hệ thống LDAP on-premises vào Amazon VPC bằng IAM. Hệ thống Identity store hiện tại KHÔNG tương thích với SAML 2.0.

✓ ĐÁP ÁN ĐÚNG

Custom Identity Broker Application kết hợp với **AWS STS (AssumeRoleWithCustomIdentity / GetFederationToken)**

💡 GIẢI THÍCH CHI TIẾT ĐÁP ÁN ĐÚNG:

Khi IdP không hỗ trợ chuẩn SAML 2.0, bạn bắt buộc phải viết 1 **Custom Identity Broker** để xác thực với LDAP, sau đó gọi API của **AWS STS** để lấy bộ temporary security credentials cấp cho user.

✗ Tại sao dùng SAML IdP SAI?

- Không thể dùng chuẩn SAML Identity Provider mặc định vì hệ thống LDAP gốc không tương thích SAML.

Câu 10: Mã hóa RDS for SQL Server (TDE vs RDS Encryption)

Chuyển SQL Server 2019 Enterprise lên RDS for SQL Server. Cấu hình CSDL tự động mã hóa dữ liệu trước khi ghi xuống ổ đĩa và tự động giải mã khi đọc dữ liệu lên bộ nhớ.

✓ ĐÁP ÁN ĐÚNG

Transparent Data Encryption (TDE) kết hợp AWS KMS

💡 GIẢI THÍCH CHI TIẾT ĐÁP ÁN ĐÚNG & PHÂN TÍCH TẠI SAO "ENABLE RDS ENCRYPTION" SAI:

- **TDE (Transparent Data Encryption):** Hoạt động ở cấp độ **Engine Database**. Tự động mã hóa/giải mã các file `.mdf`, `.ldf` ngay trên memory trước khi ghi xuống disk (đúng chính xác mô tả đề bài).
- **Tại sao "Enable RDS Encryption" SAI?** RDS Encryption mã hóa ở cấp độ **Block Storage (EBS Volume)**. Nó không phải là cơ chế mã hóa cấp Database của SQL Server và không tương thích nếu file backup từ on-premises đã dùng TDE.

Câu 11: Xử lý trùng lặp tin nhắn trong SQS (Select TWO)

Consumer xử lý chậm gây timeout làm phát sinh tin nhắn trùng lặp. Đã dùng SQS Standard Queue nhưng duplicate vẫn xảy ra. Cần đảm bảo giao tin nhắn bền vững và ngăn ngừa duplicate.

✓ 2 ĐÁP ÁN ĐÚNG

1. **Convert the SQS Standard queue to an SQS FIFO queue.**
2. **Configure MessageDeduplicationId or Content-based Deduplication on SQS FIFO queue.**

💡 GIẢI THÍCH CHI TIẾT ĐÁP ÁN ĐÚNG:

SQS FIFO Queue hỗ trợ cơ chế **Exactly-Once Processing**. Kết hợp với `MessageDeduplicationId`, SQS sẽ tự động loại bỏ các tin nhắn trùng lặp trong cửa sổ 5 phút.

✗ Tại sao các giải pháp khác SAI?

- **SQS Standard Queue:** Bản chất là *At-Least-Once Delivery*, luôn có khả năng bị duplicate tin nhắn.
- **Tăng Visibility Timeout trên Standard Queue:** Giúp Consumer có thêm thời gian nhưng không triệt tiêu được bản chất duplicate của Standard queue.

Câu 12: Tìm kiếm và Lọc dữ liệu trong AWS X-Ray (Select TWO)

Dùng AWS X-Ray tạo trace. Cần dùng Annotations trong segment document để nhận diện và lọc dữ liệu với cấu hình tối thiểu (minimal configuration).

✓ 2 ĐÁP ÁN ĐÚNG

1. Use filter expressions via the X-Ray console.
2. Fetch the trace IDs and annotations using the GetTraceSummaries API.

💡 GIẢI THÍCH CHI TIẾT ĐÁP ÁN ĐÚNG:

Annotations là các cặp Key-Value được **Đánh chỉ mục (Indexed)**. Do đó, bạn có thể lọc trực tiếp bằng Filter Expressions trên Console hoặc dùng API `GetTraceSummaries` truyền tham số lọc.

✗ Tại sao các phương án khác SAI?

- **Sampling Rules:** Dùng để điều chỉnh tỷ lệ lấy mẫu trace (sample rate), không dùng để lọc dữ liệu.
- **BatchGetTraces API:** Yêu cầu phải truyền sẵn danh sách `TraceIds`, không hỗ trợ tìm kiếm lọc theo Annotation nếu chưa biết ID.

Câu 13: Tối ưu Chi phí Kinesis Data Stream khi Shards bị Underutilized

Hệ thống Kinesis hoạt động độ trễ thấp tốt nhưng các shard được cấp phát đang bị dưới công suất (underutilized). Giải pháp nào tốt nhất để giảm chi phí và dung lượng stream?

✓ ĐÁP ÁN ĐÚNG

Merge cold shards

💡 GIẢI THÍCH CHI TIẾT ĐÁP ÁN ĐÚNG:

Cold Shards là các shard thừa thãi dung lượng. Gộp (Merge) các cold shard lại với nhau giúp giảm tổng số lượng shard trong stream, từ đó giảm trực tiếp chi phí tính theo giờ của Kinesis.

✗ Tại sao các thao tác khác SAI?

- **Split cold shards:** Tách shard làm tăng số shard $\rightarrow$ Tăng chi phí.
- **Merge hot shards:** Gộp shard đang quá tải sẽ gây nghẽn nghiêm trọng hơn.
- **Split hot shards:** Dùng khi cần tăng capacity xử lý nghẽn, làm tăng chi phí.

Câu 14: Ghi vết SQL Query xuống RDS và HTTP APIs bằng X-Ray

Ứng dụng Elastic Beanstalk cần ghi lại cuộc gọi ra RDS và HTTP APIs (bao gồm cả SQL query) và có thể tìm kiếm bằng Filter Expressions trên X-Ray Console.

✓ ĐÁP ÁN ĐÚNG

Add annotations in the subsegment section of the segment document.

💡 GIẢI THÍCH CHI TIẾT ĐÁP ÁN ĐÚNG:

- **Subsegments:** Đại diện cho các cuộc gọi downstream ra bên ngoài (RDS, External APIs).
- **Annotations:** Là dữ liệu được indexed \$ ightarrow\$ Bắt buộc phải dùng Annotation thì mới search bằng Filter Expressions trên Console được.

✗ Tại sao Metadata hoặc Segment chính lại SAI?

- **Metadata:** Không được indexed, không thể tìm kiếm bằng Filter Expressions.
- **Ghi vào Segment chính:** Segment chính đại diện cho bản thân ứng dụng web, các cuộc gọi downstream ra RDS phải nằm ở **Subsegment**.